

DA NANG UNIVERSITY OF SCIENCE AND TECHNOLOGY

BKDN.NeoArc

Nguyen Tien Hung, Nguyen Dai Duc, Nguyen Van Ngoc

October 25, 2026

Mục lục

1 Algorithms **2**
1.1 Segment Tree. 2
1.2 GCD O(1) 2
1.3 sliding window 2

2 Number theory **2**
2.1 Rabin Miller 2
2.2 Pollard's Rho 3
2.3 Sàng nguyên tố trong đoạn L, r 4
2.4 Sàng nguyên tố 4
2.5 Số Catalan 4

3 Basic Data **4**
3.1 Map 4
3.2 Binary Lifting 4
3.3 Hashing 5
3.4 Manacher 5
3.5 Aho-Corasick 5
3.6 Ham Phi Euler 5
3.7 count optimal divisors 5
3.8 Count the divisors of multiple numbers 6
3.9 Sum divisors 7
3.10 Euler totient function 7
3.11 kCn + Pow 7
3.12 nKc 7
3.13 Giai thừa modulo p 7
3.14 KMP 8
3.15 Fenwick Tree 8
3.16 Segmentree Lazy 8
3.17 LIS 9
3.18 LCA 9
3.19 Treap 9
3.20 Create test 10
3.21 Meet In The Middle (MITM) 10
3.22 Dp bitmask 10
3.23 Dp Digit sum cac chu so a -> b 11
3.24 Dp digit 1 -> k chia het cho d 11
3.25 Cặp ghép cực đại 11
3.26 Disjoint Set Union (DSU) 12
3.26.1 Union by size 12
3.26.2 Union by rank 12

4 Graphs **12**
4.1 Depth-First Search (DFS) 12
4.2 Breadth-First Search (BFS) 12
4.3 Bellman-Foyd algorithm 13
4.4 Dijkstra's algorithm 13
4.5 Floyd–Warshall algorithm 13
4.6 Disjoint Set Union Rollback 13
4.7 Sweep Line 14
4.7.1 The problem of finding the closest pair of points 14
4.7.2 Problem of finding intersection of line segments parallel to coordinate axes . . . 14
4.7.3 Problem of finding the total area covered by rectangles 15

5 Template **16**
5.1 Python3/PyPy3 16
5.2 Matrix multiplication 17
5.3 Đạo Hàm 17
5.4 Cộng 3 số lớn 18

1 Algorithms

1.1 Segment Tree.

```
/**
#include <bits/stdc++.h>
using namespace std;

const int inf  = 1e9 + 7;
const int maxN = 1e5 + 7;

int n, q;
int a[maxN];
int st[4 * maxN];

void build(int id, int l, int r) {
    if (l == r) {
        st[id] = a[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(2 * id, l, mid);
    build(2 * id + 1, mid + 1, r);
    st[id] = min(st[2 * id], st[2 * id + 1]);
}

void update(int id, int l, int r, int i, int val) {
    if (l > i || r < i) return;
    if (l == r) {
        st[id] = val;
        return;
    }
    int mid = (l + r) >> 1;
    update(2 * id, l, mid, i, val);
    update(2 * id + 1, mid + 1, r, i, val);
    st[id] = min(st[2 * id], st[2 * id + 1]);
}

int get(int id, int l, int r, int u, int v) {
    if (l > v || r < u) return inf;
    if (l >= u && r <= v) return st[id];
    int mid = (l + r) >> 1;
    int get1 = get(2 * id, l, mid, u, v);
    int get2 = get(2 * id + 1, mid + 1, r, u, v);
    return min(get1, get2);
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);
    cin >> q;
    while (q--) {
        int type, x, y;
        cin >> type >> x >> y;
        if (type == 1) update(1, 1, n, x, y);
        else cout << get(1, 1, n, x, y) << '\n';
    }
}

}
*/
```

1.2 GCD O(1)

```
int gcd(int a, int b)
{
    if (!a || !b) return a | b;
    int shift = __builtin_ctz(a | b);
    a >>= __builtin_ctz(a);
    do
    {
        b >>= __builtin_ctz(b);
        if (a > b)
            swap(a, b);
        b -= a;
    } while (b);

    return a << shift;
}
```

1.3 sliding window

```
vector<int> sldwd(vector<int> & ARR, int K, bool mx)
  deque<pair<int, int>> w;
  vector<int> ans;
  for (int i=0; i < ARR.size(); i++) {
    if (mx) {
      while (!w.empty() && w.back().first < ARR[i])
        w.pop_back();
    } else {
      while (!w.empty() && w.back().first >= ARR[i])
        w.pop_back();
    }
    w.push_back(make_pair(ARR[i], i));
    while (w.front().second <= i - K)
      w.pop_front();
    ans.push_back(w.front().first);
  }
  return ans;
}
```

2 Number theory

2.1 Rabin Miller

```
long long binary(long long a, long long k, long long n
    a = a % n; long long res = 1;
    while (k){
        if (k & 1) res = (res * a) % n;
        a = (a * a) % n;
        k /= 2;
    }
    return res;
}
bool test(long long a, long long n, long long k, long
    long long mod = binary(a, m, n);
    if (mod == 1 || mod == n - 1) return true;
    for (int l = 1; l < k; ++l){
        mod = (mod * mod) % n;
        if (mod == n - 1) return true;
    }
    return false;
}

bool RabinMiller(long long n){
    if (n == 2 || n == 3 || n == 5 || n == 7)
        return true;
{   if (n < 11) return false;
    long long k = 0, m = n - 1;
    while (m % 2 == 0){
        m /= 2;
        k++;
    }
    const static int repeatTime = 3;
    for (int i = 0; i < repeatTime; ++i){
        long long a = rand() % (n - 3) + 2;
        if (!test(a, n, k, m)) return false;
    }
```

```
    return true;
}
```

2.2 Pollard's Rho

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
ll mult(ll x, ll y, ll md) {
    ull q = (ld)x * y / md;
    ll res = ((ull)x * y - q * md);
    if (res >= md) res -= md;
    if (res < 0) res += md;
    return res;
}
ll powMod(ll x, ll p, ll md) {
    if (p == 0) return 1;
    if (p & 1)
        return mult(x, powMod(x, p - 1, md), md);
    return powMod(mult(x, x, md), p / 2, md);
}
bool checkMillerRabin(ll x, ll md, ll s, int k) {
    x = powMod(x, s, md);
    if (x == 1) return true;
    while (k--) {
        if (x == md - 1) return true;
        x = mult(x, x, md);
        if (x == 1) return false;
    }
    return false;
}

// Hàm kiểm tra nhanh một số có nguyên tố hay không
bool isPrime(ll x) {
    if (x == 2 || x == 3 || x == 5 || x == 7)
        return true;
    if (x % 2 == 0 || x % 3 == 0
     || x % 5 == 0 || x % 7 == 0)
        return false;
    if (x < 121) return x > 1;
    ll s = x - 1;
    int k = 0;
    while (s % 2 == 0) {
        s >>= 1;
        k++;
    }
    if (x < 1LL << 32) {
        for (ll z : {2, 7, 61}) {
            if (!checkMillerRabin(z, x, s, k))
                return false;
        }
    } else {
        for (ll z : {2, 325, 9375, 28178, 450775,
        9780504, 1795265022}) {
            if (!checkMillerRabin(z, x, s, k))
                return false;
        }
    }
    return true;
}

ll gcd(ll x, ll y) { return y == 0 ? x : gcd(y, x % y); }

mt19937_64 rng(chrono::steady_clock::now()
                    .time_since_epoch().count());
long long get_rand(long long r) {
    return uniform_int_distribution<long long>
                    (0, r - 1)(rng);
}

void pollard(ll x, vector<ll> &ans) {
    if (isPrime(x)) {
        ans.push_back(x);
        return;
    }
    ll c = 1;
    while (true) {
        c = 1 + get_rand(x - 1);
        auto f = [&](ll y) {
            ll res = mult(y, y, x) + c;
            if (res >= x) res -= x;
            return res;
        };
        ll y = 2;
        int B = 100;
        int len = 1;
        ll g = 1;
        while (g == 1) {
            ll z = y;
            for (int i = 0; i < len; i++) {
                z = f(z);
            }
            ll zs = -1;
            int lft = len;
            while (g == 1 && lft > 0) {
                zs = z;
                ll p = 1;
                for (int i = 0; i < B && i < lft; i++)
                    p = mult(p, abs(z - y), x);
                    z = f(z);
                }
                g = gcd(p, x);
                lft -= B;
            }
            if (g == 1) {
                y = z;
                len <<= 1;
                continue;
            }
            if (g == x) {
                g = 1;
                z = zs;
                while (g == 1) {
                    g = gcd(abs(z - y), x);
                    z = f(z);
                }
            }
            if (g == x) break;
            assert(g != 1);
            pollard(g, ans);
            pollard(x / g, ans);
            return;
        }
    }
}
vector<ll> factorize(ll x) {
    vector<ll> ans;
    for (ll p : {2, 3, 5, 7, 11, 13, 17, 19}) {
        while (x % p == 0) {
            x /= p;
            ans.push_back(p);
        }
    }
    if (x != 1) {
        pollard(x, ans);
    }
    sort(ans.begin(), ans.end());
    return ans;
}

// return pairs of (p, k) where x = product(p^k)
vector<pair<ll, int>> factorize_pk(ll x) {
    auto ps = factorize(x);
    ll last = -1, cnt = 0;
    vector<pair<ll, int>> res;
```

```
    for (auto p : ps) {
        if (p == last)
            ++cnt;
        else {
            if (last > 0) res.emplace_back(last, cnt);
            last = p;
            cnt = 1;
        }
    }
    if (cnt > 0) {
        res.emplace_back(last, cnt);
    }
    return res;
}

//apdung
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// Giả sử đã có factorize_pk(x)
extern vector<pair<ll,int>> factorize_pk(ll x);

// Tính số ước của x
ll count_divisors(ll x) {
    auto fac = factorize_pk(x);
    ll res = 1;
    for (auto [p, k] : fac) {
        res *= (k + 1);
    }
    return res;
}

int main() {
    ll n;
    cin >> n;
    cout  << count_divisors(n) << "\n";
}

// mot so truong hop
% Tính số ước, tổng ước, tích ước
% Dựa vào công thức:
% nếu \( n = p_1^{a_1} p_2^{a_2}...p_k^{a_k} \), thì:
% • Số ước: \( (a_1 + 1)(a_2 + 1)...(a_k + 1) \)
% • Tổng ước: \( \prod \frac{p_i^{a_i+1}-1}{p_i-1} \)
```

2.3 Sàng nguyên tố trong đoạn L, r

```
vector<bool> is_prime;
void sieve(int L, int R){
    is_prime.assign(R - L + 1, true);
    for (long long i = 2; i * i <= R; ++i) {
        for (long long j = max(i * i,
        (L + i - 1) / i * i); j <= R; j += i) {
            is_prime[j - L] = false;
        }
    }

    if (1 >= L) {  // Xét riêng trường hợp số 1
        is_prime[1 - L] = false;
    }

    for (long long x = L; x <= R; ++x) {
        if (is_prime[x - L]) {
            // x là số nguyên tố
        }
    }
}
```

2.4 Sàng nguyên tố

```
#include<bits/stdc++.h>
using namespace std;
// #define int long long
#define pb push_back

const int maxn = 1e5 + 5;
int a[maxn];
vector<int> min_prime, primes;
void linear_sieve(int n){
    min_prime.assign(n + 1, 0);

    for (int i = 2; i <= n; ++i) {
        if (min_prime[i] == 0) {
            min_prime[i] = i;
            primes.push_back(i);
        }
        for (int j = 0; i * primes[j] <= n; ++j) {
            min_prime[i * primes[j]] = primes[j];
            if (primes[j] == min_prime[i]) {
                break;
            }
        }
    }
}
bool is_prime(int x) {
    return x >= 2 && min_prime[x] == x;
}
int main(){
    linear_sieve(maxn);
    int n; cin >> n;
    for(int i = 1; i <= n; i++) a[i] = i;
    for(int i = 1; i <= n; i++)
        if(is_prime(a[i]))
            cout << a[i] << " ";
}
```

2.5 Số Catalan

```
int findCatalan(int n) {
    int res = 1;
    for (int i = 2; i <= n; i++) {
        res = ((res * (4 * i - 2)) / (i + 1));
    }
    return res;
}
```

3 Basic Data

3.1 Map

```
#include <iostream>
#include <algorithm>
#include <map>

using namespace std;

int main(){
    map<int, int> mp;
    map<int,int>::iterator it;
    for(it = mp.begin(); it != mp.end(); it++){
        cout << "key = " << (*it).first << ",
        value = " << (*it).second << endl;
    }
}
```

3.2 Binary Lifting

```
Node find(Node u, int d) {
  while (u.depth > d) {
      if (u.jump.depth < d) u = u.parent;
      else {
        u = u.jump;
      }
  }
```

```
  return u;
}
```

3.3 Hashing

```
typedef long long ll;
const int base = 31;
const ll MOD = 1000000003;
const ll maxn = 1000111;
using namespace std;
ll POW[maxn], hashT[maxn];
ll getHashT(int i,int j) {
    return (hashT[j] - hashT[i - 1]
    * POW[j - i + 1] + MOD * MOD) % MOD;
}
int main() {
    // Input
    string T, P;
    cin >> T >> P;
    // Initialize
    int lenT = T.size(), lenP = P.size();
    T = " " + T;
    P = " " + P;
    POW[0] = 1;
    for (int i = 1; i <= lenT; i++)
        POW[i] = (POW[i - 1] * base) % MOD;
    for (int i = 1; i <= lenT; i++)
        hashT[i] = (hashT[i - 1] *
        base + T[i] - 'a' + 1) % MOD;
    ll hashP=0;
    for (int i = 1; i <= lenP; i++)
        hashP = (hashP * base + P[i] - 'a' + 1) % MOD;
    for (int i = 1; i <= lenT - lenP + 1; i++)
        if (hashP == getHashT(i, i + lenP - 1))
            printf("%d ", i);
}
```

3.4 Manacher

```
struct binary_index_tree {
  int n;
  int t[2*N]; // assumed N is defined elsewhere

  void add(int where, long long what) {
    for (where++; where <= n; where += where & -where) {
      t[where] += what;
    }
  }

  void add(int from, int to, long long what) {
    add(from, what);
    add(to + 1, -what);
  }

  long long query (int where){
    long long sum = 0; // inferred
    for (where++; where > 0; where -= where & -where) {
      sum += t [where];
    }
    return sum;
  }
};
```

3.5 Aho-Corasick

```
struct Dsu {
  vector<int> p;

  Dsu(int n) {
    p.resize(n);
    for (int i=0; i < n; i++) {
      p[i]=i;
    }
  }

  int Find(int x) { return x == p[x] ? x :
  p[x] = Find(p[x]); }

  int Join(int x, int y) {
    int px = Find(x), py = Find(y);
    if (px == py) return 0;
    p[px]=py;
    return 1;
  }
};
```

3.6 Ham Phi Euler

```
#include <bits/stdc++.h>
using namespace std;
const int N = 5e6 + 10;
const int MOD = 998'244'353;
int phi[N];
) {
int calcPhi(int n); // tính phi hàm Euler của n
void sievePhi(int n); // sàng phi hàm Euler từ 1 đến n
int mul(int a, int b); // tính (a * b) % MOD

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int a, b; cin >> a >> b;
    sievePhi(b);
    int phiA = calcPhi(a);
    long long sum = 0;
 {  for(int i = 1; i <= b; ++i){
        int g = __gcd(a, i);
        sum = sum +
        ((1ll * phi[i] * phiA / phi[g] * g) % MOD);
        sum %= MOD;
    }
    cout << sum;
    return 0;
}

int calcPhi(int n){
    int totient = n;
    for(int i = 2; 1ll * i * i <= n; ++i){
        if(n % i == 0){
            while(n % i == 0) n /= i;
            totient -= totient / i;
        }
    }
    if(n != 1) totient -= totient / n;

    return totient;
}


void sievePhi(int n) {
    for (int i = 0; i <= n; ++i) phi[i] = i;
    for (int i = 2; i <= n; ++i) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
}

int mul(int a, int b){
    return (1ll * a * b) % MOD;
}
```

3.7 count optimal divisors

```
const int MAXN = 1e6;
```

```
vector<int> primes;
vector<bool> isPrime;

void sieve() {
    isPrime.assign(MAXN + 1, 1);
    isPrime[0] = isPrime[1] = 0;
    for (int i = 2; i <= MAXN; i++) {
        if (!isPrime[i]) continue;
        primes.push_back(i);
        for (int j = i + i; j <= MAXN; j += i) {
            isPrime[j] = 0;
        }
    }
}

long long binaryPower(long long a,
    long long k, long long n) {
    a = a % n;
    long long res = 1;
    while (k) {
        if (k & 1)
            res = (res * a) % n;
        a = (a * a) % n;
        k /= 2;
    }
    return res;
}

bool test(long long a, long long n,
    long long k, long long m) {
    long long mod = binaryPower(a, m, n);
    if (mod == 1 || mod == n - 1)
            return 1;
    for (int l = 1; l < k; ++l) {
        mod = (mod * mod) % n;
        if (mod == n - 1)
            return 1;
    }
    return 0;
}

bool isPrimeRabinMiller(long long n) {
    vector<int> checkSet = {2, 3, 5, 7,
                            11, 13, 17, 19,
                            23, 29, 31, 37};
    if (n <= 37) {
        for (int i : checkSet) {
            if (i == n) return 1;
        }
        return 0;
    }

    long long k = 0, m = n - 1;
    while (m % 2 == 0) {
        m /= 2;
        k++;
    }

    for (auto a : checkSet)
        if (!test(a, n, k, m))
            return 0;
    return 1;
}

bool isSquare(long long n){
    long long c = sqrt(n + 4);
    return c * c == n
    || (c - 1) * (c - 1) == n;
}

int countDiv(long long n) {
    if (n == 1) return 1;
    vector<int> powV;
    for (auto p : primes) {
        int cnt = 0;
        while (n % p == 0) {
            n /= p;
            ++cnt;
        }
        if (cnt) powV.push_back(cnt);
    }
    if (n != 1) {
        if (isPrimeRabinMiller(n))
            powV.push_back(1);
        else if (isSquare(n))
            powV.push_back(2);
        else {
            powV.push_back(1);
            powV.push_back(1);
        }
    }
    int ret = 1;
    for (auto i : powV) ret *= (i + 1);
    return ret;
}

void solution() {
    sieve();
    long long n;
    cin >> n;
    cout << countDiv(n);
}
```

3.8 Count the divisors of multiple numbers

```
const int MAXN = 1e6;

vector<int> minPDiv;

void sieve() {
    minPDiv.resize(MAXN + 1);
    for (int i = 2; i <= MAXN; i++) {
        if (minPDiv[i]) continue;
        minPDiv[i] = i;
        for (int j = i + i; j <= MAXN; j += i) {
            minPDiv[j] = i;
        }
    }
}

int countDiv(int n) {
    if (n == 1) return 1;
    vector<int> powV;
    int lastDiv = 0;
    int cnt = 0;
    while (n != 1) {
        if (minPDiv[n] != lastDiv) {
            if (cnt) powV.push_back(cnt);
            cnt = 0;
        }
        ++cnt;
        lastDiv = minPDiv[n];
        n /= minPDiv[n];
    }
    if (cnt) powV.push_back(cnt);
    int ret = 1;
    for (auto i : powV) ret *= (i + 1);
    return ret;
}

void solution() {
    sieve();
    int q;
    cin >> q;
    while (q--) {
        int n;
```

```
        cin >> n;
        cout << countDiv(n) << "\n";
    }
}
```

3.9 Sum divisors

```
int divSum(int n) {
    int ret = 0;
    for (int i = 1; i * i <= n; i ++) {
        ret += (i + n / i) * (n % i == 0);
        if (i * i == n) ret -= i;
    }
    return ret;
}
```

3.10 Euler totient function

```
for (int i = 1; i <= N; i++) phi[i] = i;
for (int i = 2; i <= N; i++)
    if (phi[i] == i)
       for (int j = i; j <= N; j += i)
           phi[j] -= phi[j]/i;
```

3.11 kCn + Pow

```
int binpow(int a, int n) {
    long long res = 1;
    while (n > 0) {
        if (n % 2 != 0) {
            res = (res * a) % M;
        }
        a = ((long long)a * a) % M;
        n /= 2;
    }
    return (int)res;
}
int C(int N, int K) {
    if (K > N) {
        return 0;
    }
    return (((fact[N] *
    binpow(fact[N - K], M - 2)) % M) *
    binpow(fact[K], M - 2)) % M;
}
```

3.12 nKc

```
const int MOD = 1e9 + 7;
const int N = 1e6;
int fact[N + 5], ifact[N + 5];

// Hàm lũy thừa nhanh
long long binpow(long long a, long long b) {
    long long ans = 1;
    while (b > 0){
        if (b % 2) ans = ans * a % MOD;
        a = a * a % MOD;
        b /= 2;
    }
    return ans;
}

// Chuẩn bị
void prepare(){
    // Tính fact[]
    fact[0] = 1;
    for (int i = 1; i <= N; i++)
        fact[i] = 1LL * fact[i - 1] * i % MOD;

    // Tính ifact[]
    ifact[N] = binpow(fact[N], MOD - 2);
    for (int i = N - 1; i >= 1; i--)
      ifact[i] = 1LL * ifact[i + 1] * (i + 1) % MOD;
}

// Hàm tính nCk
int C(int n, int k){
    if (k > n) return 0;
    return (1LL * fact[n] * ifact[k] % MOD)
    * ifact[n - k] % MOD;
}

int main(){
    prepare();
    // Truy vấn
    int q; cin >> q;
    while (q--){
        int n, k; cin >> n >> k;
        cout << C(n, k) << '\n';
    }
}
```

3.13 Giai thừa modulo p

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1000000; // 10^6
vector<int> primes;

// Công thức Legendre
int largestPower(int n, int p){
    int x = 0;
    while (n) {
      n /= p;
      x += n;
    }
    return x;
}

int binpow(int x, int y, int mod){
    int res = 1;
    x = x % mod;
    while (y) {
      if (y & 1)
            res = (1LL * res * x) % mod;
        y >>= 1;
        x = (1LL* x * x) % mod;
    }
    return res;
}

// Sàng nguyên tố
void prepare(){
    bool isPrime[N + 1];
    memset(isPrime, 1, sizeof(isPrime));
    isPrime[0] = isPrime[1] = 0;
    for (int i = 2; i * i <= N; i++) {
      if (isPrime[i]) {
          for (int j = i * i; j <= N; j += i) {
                isPrime[j] = 0;
            }
        }
    }
    for (int i = 2; i <= N; i++) {
      if (isPrime[i]) {
            primes.push_back(i);
        }
    }
}

int modFact(int n, int mod){
    int res = 1;
    for (int p : primes) {
      if (p <= n) {
            int k = largestPower(n, p);
            res = (1LL * res * binpow(p, k, mod)) % mo
```

```
        }
        else {
            break;
        }
    }
    return res;
}

int main(){
    prepare();
    int n, mod;
    cin >> n >> mod;
    cout << modFact(n, mod);
    return 0;
}
```

3.14 KMP

```
int longestPalSubseq(string s) {
    int n = s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) dp[i][i] = 1;
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (s[i] == s[j])
                dp[i][j] = dp[i + 1][j - 1] + 2;
            else
                dp[i][j] = max(dp[i + 1][j],
                               dp[i][j - 1]);
        }
    }
    return dp[0][n - 1];
}
```

3.15 Fenwick Tree

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct FenwickTree {
    vector<ll> s;

    FenwickTree(int n) : s(n) {}

    void update(int pos, ll dif) {
        for (; pos < (int)s.size(); pos |= pos + 1)
            s[pos] += dif;
    }

    ll query(int pos) const {
        ll res = 0;
        for (; pos > 0; pos &= pos - 1)
            res += s[pos - 1];
        return res;
    }

    ll range_query(int l, int r) const {
        return query(r) - query(l);
    }
// Tìm vị trí nhỏ nhất pos sao cho sum[0..pos]>= sum
// Trả về n nếu không có sum nào >= sum,
// hoặc -1 nếu sum <= 0
    int lower_bound(ll sum) const {
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >>= 1) {
            if (pos + pw <= (int)s.size()
            && s[pos + pw - 1] < sum) {
                sum -= s[pos + pw - 1];
                pos += pw;
            }
        }
        return pos;
    }
};
```

3.16 Segmentree Lazy

```
#pragma once
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

struct Node {
    Node *l = nullptr, *r = nullptr;
    int lo, hi;
    int mset = INF;
    int madd = 0;
    int val = -INF;

    // Khởi tạo node cho đoạn [lo, hi)
    Node(int lo, int hi) : lo(lo), hi(hi) {}

    // Xây cây từ mảng v[lo..hi)
    Node(const vector<int>& v, int lo, int hi)
    : lo(lo), hi(hi) {
        if (lo + 1 < hi) {
            int mid = (lo + hi) / 2;
            l = new Node(v, lo, mid);
            r = new Node(v, mid, hi);
            val = max(l->val, r->val);
        } else {
            val = v[lo];
        }
    }

    // Query giá trị lớn nhất trong [L, R)
    int query(int L, int R) {
        if (R <= lo || hi <= L) return -INF;
        if (L <= lo && hi <= R) return val;
        push();
        return max(l->query(L, R), r->query(L, R));
    }

    // Set toàn bộ [L, R) = x
    void set(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            mset = x;
            madd = 0;
            val = x;
        } else {
            push();
            l->set(L, R, x);
            r->set(L, R, x);
            val = max(l->val, r->val);
        }
    }

    // Cộng thêm x cho toàn bộ [L, R)
    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            if (mset != INF) mset += x;
            else madd += x;
            val += x;
        } else {
            push();
            l->add(L, R, x);
            r->add(L, R, x);
            val = max(l->val, r->val);
```

```
        }
    }

    // Đẩy lazy xuống con
    void push() {
        if (!l) {
            int mid = (lo + hi) / 2;
            l = new Node(lo, mid);
            r = new Node(mid, hi);
        }

        if (mset != INF) { // nếu có pending "set"
            l->set(lo, hi, mset);
            r->set(lo, hi, mset);
            mset = INF;
        } else if (madd) { // nếu có pending "add"
            l->add(lo, hi, madd);
            r->add(lo, hi, madd);
            madd = 0;
        }
    }
};
int main() {
    vector<int> a = {1, 3, 5, 7, 9, 11};
    Node seg(a, 0, a.size());

    cout << seg.query(1, 5) << "\n";

    seg.add(1, 4, 3);
    cout << seg.query(0, 6) << "\n";

    seg.set(2, 5, 0);
    cout << seg.query(0, 6) << "\n";
}
```

3.17 LIS

```
#include <bits/stdc++.h>
using namespace std;

vector<int> lis(const vector<int>& S) {
    if (S.empty()) return {};
    vector<int> prev(S.size());
    vector<pair<int, int>> res;
    for (int i = 0; i < (int)S.size(); i++) {
        auto it = lower_bound(res.begin(),
            res.end(), pair<int,int>{S[i], 0});
        if (it == res.end()) res.emplace_back(),
            it = res.end() - 1;
        *it = {S[i], i};
        prev[i] = (it == res.begin()) ?
            0 : (it - 1)->second;
    }
    int L = res.size(), cur = res.back().second;
    vector<int> ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
int main() {
    vector<int> a = {3, 1, 5, 2, 6, 4, 9};
    vector<int> idx = lis(a);

    cout << "Chỉ số LIS: ";
    for (int i : idx) cout << i << " ";
    cout << "\nGiá trị LIS: ";
    for (int i : idx) cout << a[i] << " ";
    cout << "\n";
}
```

3.18 LCA

```
    #include <bits/stdc++.h>
using namespace std;

struct RMQ {
    vector<vector<int>> jmp;
    RMQ(const vector<int>& V) {
        int N = V.size(), on = 1, depth = 1;
        while (on * 2 <= N) on *= 2, depth++;
        jmp.assign(depth, V);
        for (int i = 1; i < depth; i++)
            for (int j = 0; j + (1 << i) <= N; j++)
                jmp[i][j] = (jmp[i - 1][j]
                    < jmp[i - 1][j + (1 << (i - 1))])
                     ? jmp[i - 1][j] :
                     jmp[i - 1][j + (1 << (i - 1))];
    }
    int query(int a, int b) {
        int dep = 31 - __builtin_clz(b - a);
        return (jmp[dep][a] <
            jmp[dep][b - (1 << dep)])
            ? jmp[dep][a] : jmp[dep][b - (1 << dep)];
    }
};

struct LCA {
    int T = 0;
    vector<int> time, path, ret;
    RMQ rmq;

    LCA(vector<vector<int>>& C) : time(C.size()),
        rmq((dfs(C, 0, -1), ret)) {}

    void dfs(vector<vector<int>>& C, int v, int p) {
        time[v] = T++;
        for (int y : C[v]) if (y != p) {
            path.push_back(v);
            ret.push_back(time[v]);
            dfs(C, y, v);
        }
    }

    int lca(int a, int b) {
        if (a == b) return a;
        if (time[a] > time[b]) swap(a, b);
        return path[rmq.query(time[a], time[b])];
    }
};
```

3.19 Treap

```
    #pragma once
#include <bits/stdc++.h>
using namespace std;

struct Node {
    Node *l = nullptr, *r = nullptr;
    int val, y, c = 1;
    Node(int v) : val(v), y(rand()) {}
    void recalc() {
        c = (l ? l->c : 0) + (r ? r->c : 0) + 1;
    }
};

int cnt(Node* n) { return n ? n->c : 0; }

pair<Node*, Node*> split(Node* n, int k) {
    if (!n) return {nullptr, nullptr};
    if (cnt(n->l) >= k) {
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    } else {
        auto [L, R] = split(n->r,
                k - cnt(n->l) - 1);
```

```
        n->r = L;
        n->recalc();
        return {n, R};
    }
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        l->recalc();
        return l;
    } else {
        r->l = merge(l, r->l);
        r->recalc();
        return r;
    }
}

Node* insert(Node* t, Node* n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

void move(Node*& t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a, b) = split(t, l);
    tie(b, c) = split(b, r - l);
    if (k <= l) t = merge(insert(a, b, k), c);
    else t = merge(a, insert(c, b, k - r));
}
```

3.20 Create test

```
#include <bits/stdc++.h>
using namespace std;

const string NAME = "template";

const int NTEST = 100;

mt19937_64 rd(chrono::steady_clock::
now().time_since_epoch().count());
#define rand rd

long long Rand(long long L, long long R) {
    assert(L <= R);
    return L + rd() % (R - L + 1);
}

int main()
{
    srand(time(NULL));
    for (int iTest = 1; iTest <= NTEST; iTest++){
        ofstream inp((NAME + ".inp").c_str());
        inp.close();
        // Nếu dùng Linux thì "./" + Tên chương trình
        system((NAME + ".exe").c_str());
        system((NAME + "_trau.exe").c_str());
        // Nếu dùng linux thì thay fc bằng diff
        if (system(("fc " + NAME + ".out "
        + NAME + ".ans").c_str()) != 0)
        {
            cout << "Test " << iTest << ": WRONG!\n";
            return 0;
        }
        cout << "Test " << iTest << ": CORRECT!\n";
    }
    return 0;
}
```

3.21 Meet In The Middle (MITM)

```
#include <bits/stdc++.h>
using namespace std;
#define up upper_bound
#define lw lower_bound
const int N = 40 + 2;
int n, x;
int t[N];
vector<int> A, B;

void TryX(int i, int sum) {
    if (sum > x) return;

    if (i > n / 2) A.push_back(sum);
    else {
        TryX(i + 1, sum);
        TryX(i + 1, sum + t[i]);
    }
}
void TryY(int i, int sum) {
    if (sum > x) return;

    if (i > n) B.push_back(sum);
    else {
        TryY(i + 1, sum);
        TryY(i + 1, sum + t[i]);
    }
}

int main() {
    cin >> n >> x;
    for (int i = 1; i <= n; ++i) cin >> t[i];

    // Quay lui 2 tập X và Y
    TryX(1, 0);
    TryY(n / 2 + 1, 0);

    // Sắp xếp mảng B
    sort(B.begin(), B.end());

// Lặp qua mảng A và tìm kiếm nhị phân:
// - Đếm số lượng phần tử trong B
//có giá trị bằng x - A[i]
    long long cnt = 0;
    for (int sum : A) {
        cnt += up(B.begin(), B.end(), x - sum)
             - lw(B.begin(), B.end(), x - sum);
    }
    cout << cnt << '\n';
}
```

3.22 Dp bitmask

```
    const int MAXN = (1 << 15) + 1;

int n, k;
long long dp[MAXN][16];

//Lấy bit thứ k của số x
int getBit(int x, int k)
{
    return (x >> k) & 1;
}

int solve(int n, int k)
{
    for (int mask = 0; mask < (1 << n); mask++)
        for (int k = 0; k <= n; k++)
            dp[mask][k] = 0;

    //base case
    dp[0][0] = 1;
```

```
    for (int mask = 0; mask < (1 << n); mask++)
        for (int q = 1; q <= n; q++)
        {
            if (getBit(mask, q - 1)) continue;
            for (int p = 0; p <= n; p++)
            {
                //check chênh lệch của q và p
                if (p != 0 && abs(q - p) > k)
                    continue;

                //thêm q vào tập hợp
                int newMask = mask | (1 << (q - 1));

                dp[newMask][q] += dp[mask][p];
            }
        }

    long long res = 0;
    int fullMask = (1 << n) - 1;
    for (int k = 1; k <= n; k++)
        res += dp[fullMask][k];
    return res;
}
```

3.23 Dp Digit sum cac chu so a -> b

```
    #include <bits/stdc++.h>
using namespace std;
long long dp[2][20][200];
int x[20];

long long f(int idx, bool smaller, int sum);
long long G(long long X);

int main () {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    long long a, b; cin >> a >> b;
    cout << G(b) - G(a - 1);

    return 0;
}

long long f(int idx, bool smaller, int sum){
    if(idx < 0) return sum;
    long long &memo = dp[smaller][idx][sum];
    if(memo != -1) return memo;
    // tìm limit
    int lim = smaller ? 9 : x[idx];
    memo = 0;
    // điền chữ số
    for(int digit = 0; digit <= lim; ++digit){
        // chuyển trạng thái
        memo += f(idx - 1, smaller ||
            (digit < lim), sum + digit);
    }
    return memo;
}

long long G(long long X){
    // phân tích X thành các chữ số
    int n = 0; x[n] = 0;
    while(X > 0){
        x[n++] = X % 10;
        X /= 10;
    }

    memset(dp, -1, sizeof(dp));
    // chưa điền số nào => sum = 0
    return f(n - 1, 0, 0);
}
```

3.24 Dp digit 1 -> k chia het cho d

```
    #include <bits/stdc++.h>
using namespace std;
const int N = 1e4 + 10;
const int MOD = 1e9 + 7;
long long dp[2][110][N];
int x[N];
int D;
string K;

long long f(int idx, bool smaller, int sum);
long long G(string &X);

int main () {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cin >> K >> D;
    cout << G(K);

    return 0;
}
long long f(int idx, bool smaller, int sum){
    if(idx < 0) return sum == 0;
    long long &memo = dp[smaller][sum][idx];
    if(memo != -1) return memo;
    memo = 0;
    int lim = smaller ? 9 : x[idx];
    for(int i = 0; i <= lim; ++i){
        memo += f(idx - 1, smaller ||
            (i < lim), (sum + i) % D);
        memo %= MOD;
    }
    return memo;
}
long long G(string &X){
    int n = 0; x[n] = 0;
    for(int i = X.length() - 1; i >= 0; --i){
        x[n++] = X[i] - '0';
    }
    memset(dp, -1, sizeof dp);
    return (f(n - 1, 0, 0) -1 + MOD) % MOD;
}
```

3.25 Cặp ghép cực đại

```
#include <iostream>
#include <string.h>
using namespace std;

// M is number of applicants
// and N is number of jobs
#define M 6
#define N 6

// A DFS based recursive function
// that returns true if a matching
// for vertex u is possible
bool bpm(bool bpGraph[M][N], int u,
         bool seen[], int matchR[])
{
    // Try every job one by one
    for (int v = 0; v < N; v++)
    {
        // If applicant u is interested in
        // job v and v is not visited
        if (bpGraph[u][v] && !seen[v])
        {
            // Mark v as visited
            seen[v] = true;

    // If job 'v' is not assigned to an
    // applicant OR previously assigned
    // applicant for job v (which is matchR[v])
    // has an alternate job available.
```

```
    // Since v is marked as visited in
    // the above line, matchR[v] in the following
    // recursive call will not get job 'v' again
            if (matchR[v] < 0 ||
              bpm(bpGraph, matchR[v], seen, matchR))
            {
                matchR[v] = u;
                return true;
            }
        }
    }
    return false;
}

// Returns maximum number
// of matching from M to N
int maxBPM(bool bpGraph[M][N])
{
    // An array to keep track of the
    // applicants assigned to jobs.
    // The value of matchR[i] is the
    // applicant number assigned to job i,
    // the value -1 indicates nobody is
    // assigned.
    int matchR[N];

    // Initially all jobs are available
    memset(matchR, -1, sizeof(matchR));

    // Count of jobs assigned to applicants
    int result = 0;
    for (int u = 0; u < M; u++)
    {
        // Mark all jobs as not seen
        // for next applicant.
        bool seen[N];
        memset(seen, 0, sizeof(seen));

        // Find if the applicant 'u' can get a job
        if (bpm(bpGraph, u, seen, matchR))
            result++;
    }
    return result;
}

// Driver Code
int main()
{
    // Let us create a bpGraph
    // shown in the above example
    bool bpGraph[M][N] = {{0, 1, 1, 0, 0, 0},
                          {1, 0, 0, 1, 0, 0},
                          {0, 0, 1, 0, 0, 0},
                          {0, 0, 1, 1, 0, 0},
                          {0, 0, 0, 0, 0, 0},
                          {0, 0, 0, 0, 0, 1}};
    cout << maxBPM(bpGraph);
    return 0;
}
```

3.26 Disjoint Set Union (DSU)

3.26.1 Union by size

```
void make_set(int v) {
    parent[v] = v;
    size[v] = 1;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (size[a] < size[b])
            swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
}

int find_set(int v) {
    return v == parent[v] ? v :
        parent[v] = find_set(parent[v]);
}
```

3.26.2 Union by rank

```
void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

int find_set(int v) {
    return v == parent[v] ? v :
        parent[v] = find_set(parent[v]);
}
```

4 Graphs

4.1 Depth-First Search (DFS)

```
vector<int> adj[N];
bool visited[N];

void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (auto u: adj[s]) {
        dfs(u);
    }
}
```

4.2 Breadth-First Search (BFS)

```
queue<int> q;
bool visited[N];
int distance[N];

visited[x] = true;
distance[x] = 0;
q.push(x);
while (!q.empty()) {
    int s = q.front(); q.pop();
    // process node s
    for (auto u : adj[s]) {
        if (visited[u]) continue;
        visited[u] = true;
        distance[u] = distance[s]+1;
        q.push(u);
    }
}
```

4.3 Bellman-Foyd algorithm

When to use:

- Find shortest path from 1 starting node to all nodes, with edge list 'edges'
- Find if any negative cycle (run n rounds instead of n-1)

```
// Define INF
// Edge is a tuple<int,int,int> of (a, b, w)
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
for (int i = 1; i <= n-1; i++) {
    for (auto e : edges) {
        int a, b, w;
        tie(a, b, w) = e;
        distance[b] = min(distance[b],
                          distance[a]+w);
    }
}
```

4.4 Dijkstra's algorithm

When to use: Find shortest path from 1 starting node to all nodes, with adj list 'adj'

```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0,x});
while (!q.empty()) {
    int a = q.top().second; q.pop();
    if (processed[a]) continue;
    processed[a] = true;
    for (auto u : adj[a]) {
        int b = u.first, w = u.second;
        if (distance[a]+w < distance[b]) {
            distance[b] = distance[a]+w;
            q.push({-distance[b],b});
        }
    }
}
```

4.5 Floyd–Warshall algorithm

When to use: Find shortest path between all nodes, with adj matrix 'adj'

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j) distance[i][j] = 0;
        else if (adj[i][j])
            distance[i][j] = adj[i][j];
        else distance[i][j] = INF;
    }
}

for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            distance[i][j] = min(distance[i][j],
            distance[i][k]+distance[k][j]);
        }
    }
}
```

4.6 Disjoint Set Union Rollback

Cho một đồ thị G có $N \leq 2 \times 10^5$ đỉnh, và hiện tại thì chưa có cạnh nào. Có $Q (Q \leq 10^5)$ truy vấn, mỗi truy vấn có dạng *type u v* biểu thị rằng:

- Nếu $type = 1$, thêm cạnh (u, v) vào đồ thị G.
- Nếu $type = 2$, xóa cạnh (u, v) ra khỏi đồ thị G.

Sau mỗi truy vấn, hãy in ra số thành phần liên thông trong đồ thị hiện tại.

```
    struct edge
{
    int u, v;
    edge() {}
    edge(int _u, int _v) : u(_u), v(_v) {}
};

// những cạnh được update
int answer[MAXQ];
vector<edge> nw_edges[MAXQ << 2];
int sz[MAXN], root[MAXN];

struct update
{
    int node, old_size;
    update() {}
    update(int _node, int _old_size) :
    node(_node), old_size(_old_size) {}
};

stack<update> rollback;
int total_rollback[MAXQ << 2]; sau có thể rollback

int rt(int x){
    return (x == root[x] ? x : rt(root[x]));
}

// lưu lại số tplt hiện tại
int cur_comp;

void merge(int id, int x, int y)
{ // chúng ta merge như bình thường
    x = rt(x);
    y = rt(y);
    if(x == y) return;
    if (sz[x] < sz[y])
        swap(x, y);
// lưu lại size cũ của các đỉnh để rollback lại
    rollback.push({x, sz[x]});
    rollback.push({y, sz[y]});
    total_rollback[id] += 2;
    sz[x] += sz[y];
    root[y] = x;
}

// cập nhật một cạnh tồn tại trong khoảng (l, r)
void upd(int id, int l, int r, int L,
    int R, edge cur_edge){
    if (l > R || r < L)
        return;
    if (l >= L && r <= R)
    {
        nw_edges[id].push_back(cur_edge);
        return;
    }
    int mid = (l + r) >> 1;
    upd(id << 1, l, mid, L, R, cur_edge);
    upd(id << 1 | 1, mid + 1, r, L, R, cur_edge);
}

void get_ans(int id, int l, int r){
    // giả sử ta đang ở đoạn (l, r)
    // thêm vào những cạnh được lưu trong đỉnh này
    // ngoài ra lưu những node thay đổi root và size D
    for (auto e : nw_edges[id]){
        merge(id, e.u, e.v);
    }
// mỗi lần merge chúng ta cộng total_rollback[id] lên
    cur_comp -= ((int)total_rollback[id] >> 1);
    if (l == r) {
```

```
        answer[l] = cur_comp;
// đáp án cho query thứ l là số component hiện tại
    }
    // đi xuống các node con
    else{
        int mid = (l + r) >> 1;
        get_ans(id << 1, l, mid);
        get_ans(id << 1 | 1, mid + 1, r);
    }
    // update lại đáp án
    cur_comp += ((int)total_rollback[id] >> 1);
    while (total_rollback[id])
    {
        int node = rollback.top().node;
        int old_size = rollback.top().old_size;
        rollback.pop();
        root[node] = node;
        sz[node] = old_size;
        total_rollback[id]--;
    }
}
```

4.7 Sweep Line

4.7.1 The problem of finding the closest pair of points

Bài toán Khoảng cách Euclid ngắn nhất (Closest Pair of Points)

Cho một danh sách n điểm. Tìm khoảng cách Euclid ngắn nhất tạo bởi hai trong số các điểm đó.

Giới hạn:

- Số lượng điểm: $2 \leq n \leq 50000$
- Tọa độ các điểm là số nguyên: $-10^6 \leq x, y \leq 10^6$

Công thức Khoảng cách Euclid

Khoảng cách Euclid giữa hai điểm $P_i = (x_i, y_i)$ và $P_j = (x_j, y_j)$ được tính bởi công thức:

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Mục tiêu là tìm $\min_{i \neq j} d(P_i, P_j)$.

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
struct Point{
    ll x, y;
    int id;
    bool operator < (const Point& other) {
        if (x != other.x) return x < other.x;
        return y < other.y;
    }
};
struct cmp{
    bool operator ()
    (const Point& a, const Point& b) const {
        if (a.y != b.y) return a.y < b.y;
        return a.x < b.x;
    }
};

int n;
vector<Point> points; // Vector chứa tất cả các điểm
set<Point, cmp> T;

ll squared_dist(Point a, Point b) {
// Nhận vào hai điểm, trả về
// bình phương khoảng cách giữa hai điểm
    return (a.x - b.x) * (a.x - b.x) +
           (a.y - b.y) * (a.y - b.y);
}
signed main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);
    cin >> n;
    for (int i = 0; i < n; i++) {
        ll x, y; cin >> x >> y;
        points.push_back({x, y, i});
    }
    ll squared_d = squared_dist(points[0], points[1]);
// Lưu bình phương của d
    int res_id1 = 0, res_id2 = 1;
    sort(points.begin(), points.end());
// Sắp xếp các điểm theo hoành độ
    for (auto p : points) {
        ll x = p.x, y = p.y;
        int id = p.id;
        ll d = sqrt(squared_d);
        Point cur = {-1000001, y - d, id};
        while (1) {
// Tìm tất cả các điểm có tung độ trong
// khoảng [y - d, y + d]
            auto it = T.upper_bound(cur);
            if (it == T.end()) break;
            cur = *it;
            if (cur.y > y + d) break;
// Dừng lại nếu điểm có tung độ lớn hơn y + d
            if (cur.x < x - d) {
                T.erase(it);
                continue;
            }
// Xóa điểm nếu điểm này có hoành độ bé hơn x - d
            if (squared_dist(p, cur) < squared_d) {
                squared_d = squared_dist(p, cur);
                res_id1 = id; res_id2 = cur.id;
            }
// Gán đáp án mới nếu tìm được d nhỏ hơn
        }
        T.insert(p); // Thêm điểm hiện tại vào T
    }

    if (res_id1 > res_id2) swap(res_id1, res_id2);
   cout << res_id1 << " " << res_id2 << " ";
   cout << fixed << setprecision(6) << sqrt(squared_d)
}
```

4.7.2 Problem of finding intersection of line segments parallel to coordinate axes

Bài toán Đếm số giao điểm của các đoạn thẳng song song với trục tọa độ

Cho một tập hợp gồm n đoạn thẳng, mỗi đoạn thẳng song song với trục Ox hoặc trục Oy. Yêu cầu trả lại tổng số các giao điểm của tất cả các cặp đoạn thẳng trong tập hợp đó.

Giới hạn:

- Số lượng đoạn thẳng: $1 \leq n \leq 100000$
- Tọa độ x, y của mỗi điểm cuối là số thực thỏa mãn $0 \leq x, y \leq 1$

Yêu cầu chi tiết:

Các đoạn thẳng được chia thành hai loại:

- **Các đoạn thẳng ngang** (song song với trục Ox): Có dạng $y = c$ và $x \in [x_{\min}, x_{\max}]$.
- **Các đoạn thẳng đứng** (song song với trục Oy): Có dạng $x = c$ và $y \in [y_{\min}, y_{\max}]$.

Một giao điểm chỉ xảy ra giữa một đoạn thẳng ngang và một đoạn thẳng đứng khi chúng thực sự cắt nhau.

Mục tiêu:

Tính tổng số lượng cặp (i, j) đoạn thẳng $(i \neq j)$ mà doan i cat j

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 1e5 + 5;
const double EPS = 1e-9;
struct Event{
    double x;
    int y, y2, type;

    bool operator < (const Event& other) const {
        return x < other.x;
    }
};
struct FenwickTree{
    int n;
    vector<int> s;

    FenwickTree(int n) : n(n), s(n + 5) {
        for (int i = 1; i <= n; i++) s[i] = 0;
    }
    void update(int i, int val) {
        for (; i <= n; i += i & -i) s[i] += val;
    }

    int getsum(int i) {
        int res = 0;
        for (; i; i -= i & -i) res += s[i];
        return res;
    }

    int query(int l, int r) {
        return getsum(r) - getsum(l - 1);
    }
};

int n;
double blue_x1[N], blue_x2[N], blue_y[N];
double red_y1[N], red_y2[N], red_x[N];
vector<double> compress_y;
vector<Event> events;

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> blue_x1[i] >> blue_x2[i] >> blue_y[i];

        if (blue_x1[i] > blue_x2[i])
            swap(blue_x1[i], blue_x2[i]);
        compress_y.push_back(blue_y[i]);
    }

    for (int i = 1; i <= n; i++) {
        cin >> red_y1[i] >> red_y2[i] >> red_x[i];

        if (red_y1[i] > red_y2[i])
            swap(red_y1[i], red_y2[i]);
        compress_y.push_back(red_y1[i]);
        compress_y.push_back(red_y2[i]);
    }

    sort(compress_y.begin(), compress_y.end());
    compress_y.erase(unique(compress_y.begin(),
    compress_y.end()), compress_y.end());
    // Rời rạc hóa các tung độ

    for (int i = 1; i <= n; i++) {
        int yy = lower_bound(compress_y.begin(),
        compress_y.end(),
        blue_y[i]) - compress_y.begin() + 1;
        events.push_back({blue_x1[i]-EPS, yy, 0, 1});
        events.push_back({blue_x2[i]+EPS, yy, 0, 2});
    }

    for (int i = 1; i <= n; i++) {
        int l = lower_bound(compress_y.begin(),
        compress_y.end(),
        red_y1[i]) - compress_y.begin() + 1;
        int r = lower_bound(compress_y.begin(),
        compress_y.end(),
        red_y2[i]) - compress_y.begin() + 1;
        events.push_back({red_x[i], l, r, 3});
// Phần tử loại 3
    }

    sort(events.begin(), events.end());

    FenwickTree FT(n * 3);
// Có tối đa n * 3 tung độ khác nhau
    ll res = 0;
    for (auto e : events) {
        if (e.type == 1) FT.update(e.y, 1);
        else if (e.type == 2) FT.update(e.y, -1);
        else res += FT.query(e.y, e.y2);
    }

    cout << res;
}
```

4.7.3 Problem of finding the total area covered by rectangles

Bài toán Tính tổng diện tích phủ bởi các hình chữ nhật (Area of Union of Rectangles)

Trên mặt phẳng tọa độ, ta vẽ N hình chữ nhật có các cạnh song song với trục tọa độ. Tính tổng diện tích bị che phủ bởi N hình chữ nhật này (tổng diện tích của hợp các hình chữ nhật).

Giới hạn:

- Số lượng hình chữ nhật: $1 \leq n \leq 10000$
- Mỗi hình chữ nhật có tọa độ góc trái dưới và góc phải trên lần lượt là (x_1, y_1) và (x_2, y_2) sao cho $0 \leq x_1 < x_2 \leq 30000$ và $0 \leq y_1 < y_2 \leq 30000$.

Yêu cầu chi tiết:

Cho N hình chữ nhật $R_1, R_2, \ldots, R_N$. Tính diện tích của hợp (union) của chúng:

$$Area = Diżnt \cdot ch\left(\bigcup_{i=1}^{N} R_i\right)$$

Lưu ý: Các phần diện tích bị chồng lấn (overlap) chỉ được tính một lần.

```
    #include <bits/stdc++.h>

using namespace std;

const int MX = 30000;

struct Segment{
    int x, y1, y2, type;
```

```
    bool operator < (const Segment& other) const {
        return x < other.x;
    }
};

struct SegmentTree{
    vector<pair<int, int>> s;

    SegmentTree(int n) : s(n * 4 + 5) {
        for (int i = 1; i <= 4 * n; i++)
            s[i] = {0, 0};
    }

    void update(int id, int l, int r,
            int tl, int tr, int val) {
        if (l > tr || r < tl) return;
        if (l >= tl && r <= tr) {
            s[id].second += val;
            if (s[id].second != 0)
                s[id].first = r - l + 1;
            else if (l != r)
                s[id].first = s[id * 2].first
                + s[id * 2 + 1].first;
            else s[id].first = 0;
            return;
        }

        int m = (l + r) >> 1;
        update(id * 2, l, m, tl, tr, val);
        update(id * 2 + 1, m + 1, r, tl, tr, val);

        if (s[id].second != 0)
            s[id].first = r - l + 1;
        else
            s[id].first = s[id * 2].first + s[id * 2
    }
};

int n;
vector<Segment> segments;

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(NULL);

    cin >> n;
    for (int i = 1; i <= n; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        segments.push_back({x1, y1, y2, 1});
        segments.push_back({x2, y1, y2, -1});
    }

    sort(segments.begin(), segments.end());

    SegmentTree ST(MX);
    int res = 0;
    for(int i = 0;i < (int)segments.size()-1;i++){
        ST.update(1, 0, MX, segments[i].y1,
        segments[i].y2 - 1, segments[i].type);
        res += (segments[i + 1].x - segments[i].x)
        * ST.s[1].first;
    }
    cout << res;
}
```

5 Template

5.1 Python3/PyPy3

```
import sys
import math
import heapq
from collections import deque, defaultdict, Counter
sys.setrecursionlimit(2000000)

# --- Fast I/O ---
# Re-define input() to read lines faster
input = sys.stdin.readline
# Re-define print() to write to stdout faster
def print(s):
    sys.stdout.write(str(s) + "\n")
# ------------------
+ 1].first;
# --- Constants ---
MOD = 1_000_000_007
INF = float('inf')

def solve():
# Some functions & operations
# +, -, *, / (true div), // (int div), **
# math.ceil, math.floor, math.round
# math.factorial, math.gcd, math.lcm
# for index, x in enumerate(arr):
# input handling: split(), strip(), map(), list()
# format string: f"{self.p}/{self.q}"
# OOP special methods: init, repr (string
# representation), del, add, sub, mul, truediv,
# floordiv, mod, pow, len, iter
    pass

def main():
    t = int(input())

    for _ in range(t):
        solve()

if __name__ == "__main__":
    main()
```

Tóm tắt: Các kiểu dữ liệu cần thiết trong C++

1. Số nguyên

Kiểu	Kích thước	Giá trị	Ghi ch
`int`	32-bit	$\pm 2\times10^9$	Dùng c
`long long`	64-bit	$\pm 9\times10^{18}$	Phổ biế
`__int128`	128-bit	rất lớn	Tránh t
`unsigned long long`	64-bit	$0 \to 1.8\times10^{19}$	Dùng c

2. Số thực

Kiểu	Độ chính xác	Ghi chú
`double`	$\approx 10^{-9}$	Nhanh, sai số nhỏ
`long double`	$\approx 10^{-18}$	Hình học chính xác cao

3. Container chính

Kiểu	Mục đích	Độ phức tạp
`vector`	Mảng động	Truy cập $O(1)$
`set/map`	Cây đỏ-đen	$O(\log n)$
`unordered_map`	Hash table	$O(1)$ trung bình
`priority_queue`	Hàng đợi ưu tiên	Dijkstra, Greedy
`bitset<N>`	Dãy bit cố định	DP bitmask

4. Dữ liệu nâng cao

- `complex<long double>` – dùng trong FFT/DFT.
- `boost::multiprecision::cpp_int` – xử lý BigInt.
- `pair`, `tuple`, `struct` – gom nhóm dữ liệu.

5. Modular Arithmetic (mẫu ngắn)

```
const long long MOD = 1e9+7;
struct mint {
  long long x;
  mint(long long v=0){x=v%MOD; if(x<0)x+=MOD;}
  mint operator+(mint b){return mint(x+b.x);}
  mint operator*(mint b){return mint(x*b.x%MOD);}
};
```

6. Quy tắc vàng

- Có phép nhân ⇒ dùng `long long` hoặc `__int128`.
- Chia số thực ⇒ dùng `long double`.
- Trạng thái/bitmask ⇒ dùng `bitset`.
- Dữ liệu động ⇒ `vector`, `set`, `priority_queue`.
- Với $n > 10^6$: tránh $O(n^2)$, ưu tiên cấu trúc $O(\log n)$.

5.2 Matrix multiplication

```
#include <bits/stdc++.h>

using namespace std;

using type = int;
// Kiểu dữ liệu các phần tử của ma trận

struct Matrix {
    vector <vector <type> > data;

    // Số lượng hàng của ma trận
    int row() const { return data.size(); }

    // Số lượng hàng của ma trận
    int col() const { return data[0].size(); }
    auto & operator [] (int i) { return data[i]; }
    const auto & operator[] (int i)
        const { return data[i]; }
    Matrix() = default;
    Matrix(int r, int c):
    data(r, vector <type> (c)) { }
    Matrix(const vector <vector <type> > &d): data(d){
        /**/ assert(d.size());
        /**/ int size = d[0].size();
        /**/ assert(size);
        /**/ for (auto x : d) assert(x.size() == size);
    }
    // In ra ma trận.
    friend ostream & operator
    << (ostream &out, const Matrix &d) {
        for (auto x : d.data) {
            for (auto y : x) out << y << ' ';
            out << '\n';
        }
        return out;
    }

    // Ma trận đơn  vị
    static Matrix identity(long long n) {
        Matrix a = Matrix(n, n);
        while (n--) a[n][n] = 1;
        return a;
    }

    // Nhân ma trận
    Matrix operator * (const Matrix &b) {
        Matrix a = *this;

        // Kiểm tra điều kiện nhân ma trận
        assert(a.col() == b.row());

        Matrix c(a.row(), b.col());
        for (int i = 0; i < a.row(); ++i)
            for (int j = 0; j < b.col(); ++j)
                for (int k = 0; k < a.col(); ++k)
                    c[i][j] += a[i][k] * b[k][j];
        return c;
    }

    // Lũy thừa ma trận
    Matrix pow(long long exp) {
// Kiểm tra điều kiện lũy thừa ma trận
// (là ma trận vuông)
        assert(row() == col());
        Matrix base = *this, ans = identity(row());
        for (; exp > 0; exp >>= 1,
                base = base * base)
            if (exp & 1) ans = ans * base;
        return ans;
    }
};
int main(){
    Matrix a({
        {1, 2},
        {3, 4}
    });
    Matrix b({
        {0, 10, 100},
        {1,  1,  10}
    });
    cout << a * b << '\n';
    // 2 12 120
    // 4 34 340
    cout << a.pow(3) << '\n';
    // 37 54
    // 81 118
    b = a;
    cout << b << '\n';
    // 1 2
    // 3 4
    b = Matrix::identity(3);
    cout << b << '\n';
    // 1 0 0
    // 0 1 0
    // 0 0 1
    b = Matrix(2, 3);
    cout << b << '\n';
    // 0 0 0
    // 0 0 0

    Matrix c(3, 2);
    cout << c << '\n';
    // 0 0
    // 0 0
    // 0 0
}
```

5.3 Đạo Hàm

```
//F''(x)
vector<int> F2(MAX_H + 1);  // F2[x] = F''(x)
for (int i = 0; i < N; ++i) {
    // f''i(x) = 2*C(i) at x = H(i)+1
    F2[H[i] + 1] += 2 * C[i];
}
//F'(x)
vector<int> F1(MAX_H + 1);  // F1[x] = F'(x)
// Tính F'(0)
for (int i = 0; i < N; ++i) {
    F1[0] -= C[i];
}
// Tính F'(x)
for (int x = 1; x <= MAX_H; ++x) {
    F1[x] = F1[x-1] + F2[x];
}
//F(x)
```

```
vector<int> F(MAX_H + 1);
// Tính F(0)
for (int i = 0; i < N; ++i) {
    F[0] += H[i] * C[i];
}
// Tính F(x)
for (int x = 1; x <= MAX_H; ++x) {
    F[x] = F[x-1] + F1[x];
}
```

5.4 Cộng 3 số lớn

```
a, b, c = map(int, input().split())
print(a+b+c)
```